# Datenbanktypen

Joern Ploennigs · AI4SC



> Eventually, you have to throw the data over a wall.
>
> — Pat Helland

## Hörsaalfrage: Wann reichen Dateien nicht?

Diese Sitzung baut auf `12a_Datenhaltung` auf: Dort wurde sichtbar, warum dateibasierte Speicherung bei wachsenden Datenmengen, mehreren Nutzern und komplexen Abfragen an Grenzen stößt. Hier folgt nun der nächste Schritt: Welche Art von Datenbank passt zu welchem Problem?

Die Leitidee dieser Vorlesung lautet: Nicht „Welche Datenbank ist modern?“, sondern „Welches Informationsmodell beschreibt unser Problem am besten?“. Tabellarische Daten sprechen für relationale Datenbanken, Dokumentstrukturen für dokumentenorientierte Systeme und Netzwerkprobleme für Graphdatenbanken.

## Dateien kippen irgendwann um

Eine Datei kann lange genügen, solange ein einzelnes Programm oder eine Person damit arbeitet. Kritisch wird es, sobald mehrere Beteiligte gleichzeitig schreiben, verschiedene Kopien im Umlauf sind oder gezielte Auswertungen über große Datenmengen nötig werden. Dann entstehen genau die Probleme, die Dateisysteme nicht von sich aus lösen: Versionskonflikte, Redundanz, fehlende Integritätsregeln und umständliche Abfragen.

## Warum Datenbanken?

Stellt euch eine gemeinsame Tabelle mit Sensoren, Messpunkten und Prüfberichten vor. Solange nur eine Person ergänzt und niemand quer über mehrere Listen suchen muss, ist das praktikabel. Sobald aber verschiedene Anwendungen dieselben Daten lesen, Beziehungen zwischen Objekten geprüft werden sollen oder gleichzeitige Änderungen sicher bleiben müssen, reicht Dateiablage nicht mehr aus. Dann trennt man die Datenhaltung von den Programmen und übergibt Verwaltung, Zugriffskontrolle und Konsistenz einem Datenbankmanagementsystem.

## Datenbank und Datenbanksystem

Die Begriffe werden oft vermischt, bezeichnen aber unterschiedliche Ebenen. Die *Datenbank* sind die logisch zusammengehörenden Daten selbst. Das *DBMS* ist die Software, die diese Daten organisiert, schützt und zugreifbar macht. Erst beides zusammen bildet das *Datenbanksystem*, also die fachliche Datenbasis plus die technische Verwaltungsinstanz.

## Dateien versus Datenbanken

Eine Datei mit Bauteillisten, eine zweite mit Messwerten und eine dritte mit Prüfberichten kann funktionieren. Aber schon die einfache Frage „Welche Messwerte gehören zu Bauteilen mit überfälliger Prüfung?“ wird dann mühsam, weil jede Anwendung ihre eigene Struktur stillschweigend voraussetzt. Bei einer Datenbank wird dieses Informationsmodell explizit beschrieben und vom DBMS durchgesetzt. Integrität, Synchronisation und leistungsfähige Abfragen werden so zu Eigenschaften des Systems und nicht zu Einzellösungen in jedem Programm.

## Codd: Was eine Datenbank leisten soll

Diese Punkte machen deutlich, dass eine Datenbank mehr ist als ein größerer Dateiordner. Sie soll Daten integrieren statt vervielfachen, mehrere Sichten auf denselben Bestand erlauben, Widersprüche verhindern und parallele Änderungen beherrschbar machen. Erst durch solche Eigenschaften wird Datenhaltung für reale Anwendungen robust.

## Hörsaalfrage: Netzwerkproblem in Tabellen?

Diese Frage zielt auf eine typische Modellierungsfalle. Wenn das Problem selbst schon aus Verbindungen, Nachbarschaften und Pfaden besteht, ist es oft ungünstig, diese Struktur nur indirekt in Tabellen nachzubauen. Dann sollte zuerst geprüft werden, ob ein Graphmodell die natürlichere Beschreibung ist.

## Die eigentliche Auswahlfrage

Die interessante Frage lautet nicht, ob relationale oder NoSQL-Systeme „besser“ sind. Die bessere Frage ist: Welche Struktur haben unsere Daten und welche Operationen brauchen wir darauf? Diese Sicht verbindet die Datenbankwahl mit den Problemklassen aus Block 5.

## Das Informationsmodell führt

Der wichtigste Auswahlgrundsatz lautet: Erst die Struktur des Problems bestimmen, dann die Technologie benennen. Das spart Umwege, weil nicht versucht wird, jede Aufgabe in dasselbe Werkzeug zu pressen. Datenbanktypen sind also keine Moden, sondern Antworten auf verschiedene Informationsmodelle.

## Vom Problem zur Datenbank

Für ein Rohrnetz oder Verkehrsnetz ist nicht die spätere Software zuerst entscheidend, sondern die Form der Information: Knoten, Kanten und Pfade. Wenn genau diese Beziehungen fachlich im Mittelpunkt stehen, ist eine Graphdatenbank oft die stimmigere Wahl als eine tabellarische Abbildung mit vielen Hilfskonstruktionen.

## Noch zwei typische Zuordnungen

Auch außerhalb klassischer Netzprobleme hilft die Zuordnung über das Informationsmodell. BIM-nahe Objekte oder Projektdokumente sind oft verschachtelt und nicht völlig gleichförmig aufgebaut; dafür passen Document Stores gut. Verwaltungsdaten mit klaren Feldern, stabilen Beziehungen und dem Bedarf nach Konsistenzregeln bleiben dagegen ein typisches Feld relationaler Systeme.

## Datenbankfamilien

Über die Jahrzehnte haben sich mehrere Datenbankfamilien herausgebildet. Sie unterscheiden sich nicht primär durch Marketing oder Popularität, sondern durch das zugrunde liegende Datenmodell. Im Folgenden wird jede Familie mit ihrem typischen Informationsmodell und einem passenden Anwendungsfall aus dem Ingenieurkontext verknüpft. Die eingeblendete SVG-Grafik dient dabei nur als Orientierung: Sie ist eine interne Lehrgrafik und soll vor allem sichtbar machen, dass relationale Systeme weiterhin stark sind, während mehrere NoSQL-Familien daneben spezialisierte Rollen einnehmen.

### Relationale Datenbanken

Relationale Systeme sind die richtige Wahl, wenn die Daten klar strukturiert und die Beziehungen zwischen Tabellen stabil modellierbar sind. Sie sind besonders stark bei Integritätsregeln, Transaktionen und deklarativen Abfragen mit SQL.

### Dokumentenorientierte Datenbanken

Document Stores eignen sich, wenn Objekte unterschiedlich aufgebaut sein dürfen oder wenn hierarchische Strukturen natürlicher sind als Tabellen. Das ist etwa bei BIM-bezogenen Objekten, Dokumentmetadaten oder variablen Projektbeschreibungen nützlich.

### Key-Value-Datenbanken

Key-Value-Stores reduzieren Datenhaltung auf eine sehr einfache Abbildung. Das ist leistungsfähig, wenn ein eindeutiger Schlüssel bekannt ist und komplexe Struktur keine zentrale Rolle spielt. Für analytische Fragestellungen sind sie meist nicht die erste Wahl.

### Wide-Column-Datenbanken

Wide-Column-Systeme liegen zwischen einfachem Key-Value-Zugriff und stärker strukturierter Tabellenwelt. Sie eignen sich, wenn extrem viele Datensätze verteilt gespeichert werden und sich Attribute gruppenweise organisieren lassen, etwa bei Telemetrie- oder Plattformdaten in großem Maßstab.

### Time-Series-Datenbanken

Zeitreihendatenbanken sind spezialisiert auf fortlaufende Messwerte mit zeitlicher Ordnung. Typische Operationen sind Fensterbildung, Verdichtung, Grenzwertüberwachung und Trendanalyse. Für Labor-, Umwelt- oder Maschinendaten ist dieses Modell oft viel natürlicher als allgemeine Tabellen.

**Suchmaschinen-Datenbanken**
Suchorientierte Systeme sind sinnvoll, wenn große Mengen an Textdokumenten durchsuchbar gemacht werden müssen. Für Umwelt- oder Bauprojekte kann das die Recherche in Berichten, Gutachten und Protokollen stark vereinfachen.

**Graphdatenbanken**
Graphdatenbanken passen dort, wo die Beziehungen selbst der Kern des Problems sind. Das gilt für Verkehrsnetze, Leitungsnetze, Abhängigkeitsgraphen oder Empfehlungsstrukturen. Die Brücke zu `10a_Graphprobleme` ist hier direkt: Graphproblem und Graphdatenbank passen strukturell zusammen.

**Vector-Datenbanken**
Vektordatenbanken werden interessant, wenn Objekte als numerische Einbettungen vorliegen und nicht exakte Gleichheit, sondern Ähnlichkeit gesucht wird. Das ist vor allem bei KI-Anwendungen relevant, etwa für semantische Suche, Retrieval-Augmented Generation oder das Auffinden ähnlicher Dokumente und Bauteilbeschreibungen.

## Agentischer Prüfpunkt
Wenn KI ein Werkzeug empfiehlt, sollte nicht sofort die Produktfrage im Vordergrund stehen. Zuerst wird das Informationsmodell explizit gemacht, dann wird die Passung zwischen Problemstruktur und Datenbanktyp geprüft, und erst zum Schluss lohnt sich die Frage nach einem konkreten Produkt. So wird aus einer schnellen Empfehlung eine nachvollziehbare Entscheidung.

## Agentische Verifikation
Gerade bei KI-gestützten Technologieentscheidungen ist Skepsis wichtig. Ein Modell kann eine populäre Technologie nennen, ohne zu prüfen, ob sie zur Problemstruktur passt. Deshalb sollte die erste Rückfrage immer lauten: „Welches Informationsmodell nehmen wir eigentlich an?“ Erst dann ist eine fundierte Datenbankwahl möglich. Ebenso wichtig ist der Hinweis, dass polyglotte Persistenz kein Sonderfall ist: Ein System kann völlig sinnvoll eine relationale Datenbank für Stammdaten, eine Suchmaschine für Dokumente und eine Zeitreihendatenbank für Messwerte kombinieren.

Beispiel für die Verifikation in fünf Zeilen:

- Problem: Sensornetz mit Stammdaten, Messreihen und Wartungsberichten
- KI-Vorschlag: „Nehmt einfach PostgreSQL für alles.“
- Passung: Stammdaten passen gut in relationale Tabellen.
- Mismatch: Zeitreihen und Volltextsuche werden dadurch unnötig schwergewichtig.
- Bessere Antwort: Relationale DB + Time-Series- oder Suchsystem mit klarer Aufgabentrennung.

## Auswahlkriterien für diese Vorlesung
In dieser Vorlesung steht deshalb nicht das Auswendiglernen von Produktnamen im Mittelpunkt. Wichtiger ist, Entwurf und Modellierung vor die Implementierung zu stellen. Wer versteht,

welche Struktur ein Problem hat, kann später auch neue Technologien besser einordnen und bei Bedarf mehrere Systeme sinnvoll kombinieren.

### Merksätze

Dateien sind ein sinnvoller Start, aber kein Ersatz für systematische Datenhaltung unter Last, Parallelität und Integritätsanforderungen. Die zentrale Auswahlfrage lautet immer, welches Informationsmodell das Problem trägt. Daraus folgt der passende Datenbanktyp und erst danach die konkrete Technologie.

## Bibliographie